

Documentation: Attempt to deploy GTCRN Speech Enhancement on Rpi Pico using TFLM

Lakshmeesh K P

Objective:

To deploy a speech enhancement model (StreamGTCRN) using TFLM in an resource constrained MCU like raspberry pi Pico. This requires converting the model to .tflite format and performing a full INT8 quantization.

Several models available online were briefly evaluated to see which would be feasible and worthwhile to try deploying on pico. RNNoise and GTCRN seemed like the only two best models.

All the major bugs and their workarounds are mentioned in the document.

Rnnoise:

RNNoise was considered since the INT8 quantised TFLite model was already available online, which simplifies deployment. [\[1\]](#)

- It works on 10ms audio frames. It relies heavily on both DSP and a RNN to clean out noise.
- First a biquad high pass filter is applied to remove low frequency noise and DC offset.
- Broadly, we compute the band energies for each frame and apply DCT on the log spectrum (22 barkfrequency cepstral coefficients), These can be tuned to adjust broad ranges of frequencies together, but we cannot deal with the finer details with this alone. It may suppress the noise along with the speech. Narrow windows of speech can be let through by pitch filtering (6 pitch features).
- The first and second derivative wrt time of first 6 BFCCs and pitch period a spectral non-stationarity metric (helpful in speech detection) are also included ; thus totally 42 features are given as input to the GRU model. It returns 22 gain values and are applied to each of the frequency bands, and then pitch filtering is performed.
- Then the audio is reconstructed, frame wise. It is also ensured that the gains between the frames doesn't change unnaturally.
- Adaptations tried:
 - o The pitch filter stage seemed to be computationally expensive (involved auto-correlations and many multiplications). I had a doubt whether real-time would be feasible. A possible approximation would have been to avoid pitch filtering entirely. This would bring down computation significantly at the cost of quality.
 - o Before I made DMA work, only 16Khz sampling was working so I made an effort to convert the model from 48Khz to 16Khz sampling. However, this would require readjusting the barkscale frequency boundaries (eband5ms array) and also retrain the entire model.
- However, many part of the pre-processing C codes were specific for PC implementation. Thus, they couldn't be used directly and required re-writing them to make it MCU compatible. This would take a significant amount of time and did not seem to align with the objectives of BTP-2 and hence discontinued.

GTCRN (gated temporal convolutional recurrent network):

- On the github page, only the .onnx model was available. It was generated from an initial pytorch model script. I first made attempts to directly convert from onnx to tflite, and faced many compatibility issues.
- ONNX to TFlite conversion:
 - o Onnx2tf python library: Faced a lot of library resolution and version dependency related issues. Then finally, faced a dynamic dimension related issue, no channel re-arrangement options made it work.
 - o Onnx and onnx-tf python library: similarly library/version resolution issues. Dimension mismatch issue during matrix multiplication.
 - o Emmtrix onnx-to-C converter: dimension mismatch issue from GRU units for larger model. GRU bidirectional unsupported issue for simplified gtcn model.
 - o All the above issues hinted that we needed to redesign the model, to allow the conversion to happen.
- PyTorch to Tflite conversion:
 - o It was suggested to directly use LiteRT to convert from the original pytorch model to tflite.
 - o It is required to run this converter on a linux environment for library availability. (thus, done on WSL)
 - o Converter issues:
 - XLA compiler can't handle grouped convolutions (groups>1 is not supported) in ConvTranspose2d. workaround done was to replace it with 2 single custom convolution blocks made using nn.conv2d with split inputs.
 - Error in ConvBlock() from nn.ConvTranspose2d, due to upsampling for stride>1. Reimplemented it manually using nn.Conv2d.
 - o Solving the above two issues, allowed successful conversion to tflite. The .tflite file size was 1685KB. Now we need to fully INT8 quantise to deploy using TFLM on pico. Expected file size after successful quantisation would be ~420KB.
 - o Quantisation issues:
 - Issue from Select_V2 in StreamEncoder sub-node.(Node number 94 (SELECT_V2) failed to prepare.). It's most likely from slicing of caches (conv_cache, tra_cache) after checking the model flow graph on Netron. Select_V2 is essentially an "if" condition. Reason to believe so is because during INT8 quantisation we are required to pass a representative dataset through the entire model to find out output ranges. But, while slicing, an "if" condition is imposed based on index value. The model can undergo dynamic quantisation (which doesn't require representative dataset) without any problem (can be deployed on higher end microcontrollers with sufficient flash/RAM and float hardware)
 - The speculated solution is to slice the caches external to the model and feed them separately into the layers.
 - o Alternative solutions:
 - Shift to the non-streaming model, which doesn't use caches. A real-time streaming version can be made by overlapping and adding frames. However, it inputs 16k samples to the model, which makes it memory expensive.
 - Additionally Cut down the width/depth of convolutions
 - Use TCN (temporal conv network) instead of Bidirectional GRU. However, it can only remember for a fixed duration.

Work done after BTP-2, at home:

1. node 94 Select_V2 issue:

After checking the model flow graph on Netron, its most likely from slicing of recurrent caches.

To overwrite sections of the cache, the compiler might generate the SELECT_V2 to mask parts of the cache. However, during the quantisation calibration broadcast, it crashes for some reason.

Old StreamEncoder() forward script:

```
def forward(self, x, conv_cache, tra_cache):
    """
    x: (B,C,T,F)
    conv_cache: (B,C, (kT-1)*8, F)
    tra_cache: (3,1,B,C)
    """
    en_outs = []
    for i in range(2):
        x = self.en_convs[i](x)
        en_outs.append(x)

    x, conv_cache[:, :, :2, :] = self.en_convs[2](x, conv_cache[:, :, :2, :], tra_cache[0]); en_outs.append(x)
    x, conv_cache[:, :, 2:6, :] = self.en_convs[3](x, conv_cache[:, :, 2:6, :], tra_cache[1]); en_outs.append(x)
    x, conv_cache[:, :, 6:16, :] = self.en_convs[4](x, conv_cache[:, :, 6:16, :], tra_cache[2]); en_outs.append(x)

    return x, en_outs, conv_cache, tra_cache
```

New StreamEncoder() forward script:

```
def forward(self, x, c_en1, c_en2, c_en3, t_en1, t_en2, t_en3):
    """
    Inputs are now discrete cache chunks.
    c_en1 replaces conv_cache[:, :, :2, :]
    c_en2 replaces conv_cache[:, :, 2:6, :]
    c_en3 replaces conv_cache[:, :, 6:16, :]
    """
    en_outs = []
    for i in range(2):
        x = self.en_convs[i](x)
        en_outs.append(x)

    #no slicing happens here now
    x, out_c_en1, out_t_en1 = self.en_convs[2](x, c_en1, t_en1); en_outs.append(x)
    x, out_c_en2, out_t_en2 = self.en_convs[3](x, c_en2, t_en2); en_outs.append(x)
    x, out_c_en3, out_t_en3 = self.en_convs[4](x, c_en3, t_en3); en_outs.append(x)

    out_c_caches = (out_c_en1, out_c_en2, out_c_en3)
    out_t_caches = (out_t_en1, out_t_en2, out_t_en3)

    return x, en_outs, out_c_caches, out_t_caches
```

Now instead of slicing, it has been split into three and fed as input to the forward(). Then its fed to another set of output caches.

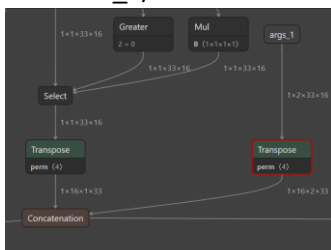
Similar changes done to StreamDecoder(), and StreamGTCRN(). (shared on github)

The corresponding changes were also made on the LiteRT converter AND quantizer python script (litert_conv_SliceOutside)

2. Conversion failed: tensorflow/lite/kernels/transpose.cc:52 op_context->perm->dims->data[0] != dims (4 != 3)Node number 43 (TRANSPOSE) failed to prepare.

This was the corresponding node as checked on netron:

stream.gtcn_stream.StreamGTCRN/stream.gtcn_stream.StreamEncoder_encoder/stream.gtcn_stream.StreamGTC onvBlock_2/modules.convolution.StreamConv2d_depth_conv; input: tensor: float32[1,16,2,33]



The suspected problem is that the converter aggressively squeezes dimensions (where the weight=1) to save space. It tries to apply a 4d “transpose” on this 3d weight to convert between NCHW and NHWC formats, causing the crash.

A finding is that TFLite’s quantizer doesn’t support NCHW (no. of batches, channels, height, width) format. It expects NHWC format data. Thus, it inserts a transpose layer in between to convert from NCHW to NHWC and then apply the

corresponding compatible ops, then transpose back to NCHW.

While this keeps the model from crashing, it can add a lot more redundant layers which can slow down the inference speed.

Original version (v1):

During the concatenation step, the tool adds transpose and also squeezes dimensions, causing the error.

V2:

We try forcing the dimensions to be fixed, and avoid squeezing using `.view()`. We convert the data format on our own and try to ensure that no unintentional dimension squeezing happens.

However, the converter squeezed the dimensions during the `.cat()` operation, resulting in a new crash.

3. Conversion failed: tensorflow/lite/kernels/concatenation.cc:200 t->dims->size != t0->dims->size (4 != 3)Node number 43 (CONCATENATION) failed to prepare.

Tried doing the `.view()` trick again with concat to avoid squeezing, but again resulted in the same transpose node error.

It was resulting in an error loop, so had to try something else.

The batch=1 dimension gets dropped no matter what.

So, then I tried to flatten the tensor from 4d to 3d, before its fed to the concat operation and then revert to 4d after it. This idea helped bypass the major dimension mismatch (4!=3) crashes. There were some more bugs to be resolved after this though.

4. Conversion failed: tensorflow/lite/kernels/reshape.cc:94 num_input_elements != num_output_elements (16 != 1056)Node number 43 (RESHAPE) failed to prepare.

This was apparently because of a reordering issue of the model inputs. If you check the original input tensor dimensions, we can see that [1,16,2,33] was probably fed instead of [1,1,16] tensor. It was suggested to change the representative dataset from a list to a dictionary and map the tensors properly.

5. Conversion failed: 'args_5'

This new error was at a much further node (around node 250).

Args_5 corresponds to t_en2. It was suggested to change the naming to directly match "args_5" from the input label "t_en2".

This resulted in the creation of a .tflite file atleast.

```
Tracing, quantizing, and converting directly to INT8 LiteRT...
W0000 00:00:1780678290.410543    904 tf_tfl_flatbuffer_helpers.cc:364] Ignored output_format.
W0000 00:00:1780678290.410600    904 tf_tfl_flatbuffer_helpers.cc:367] Ignored drop_control_dependency.
I0000 00:00:1780678290.411487    904 reader.cc:83] Reading SavedModel from: /tmp/tmpzjvlb0ho
I0000 00:00:1780678290.415468    904 reader.cc:52] Reading meta graph with tags { serve }
I0000 00:00:1780678290.415549    904 reader.cc:147] Reading SavedModel debug info (if present) from: /tmp/tmpzjvlb0ho
I0000 00:00:1780678290.441803    904 mlir_graph_optimization_pass.cc:437] MLIR V1 optimization pass is not enabled
I0000 00:00:1780678290.448775    904 loader.cc:236] Restoring SavedModel bundle.
I0000 00:00:1780678290.678875    904 loader.cc:220] Running initialization op on SavedModel bundle at path: /tmp/tmpzjvlb0ho
I0000 00:00:1780678290.737290    904 loader.cc:471] SavedModel load for tags { serve }; Status: success: OK. Took 325822 microseconds.
I0000 00:00:1780678290.808882    904 dump_mlir_util.cc:269] disabling MLIR crash reproducer, set env var 'MLIR_CRASH_REPRODUCER_DIRECTORY' to enable.
I0000 00:00:1780678362.318256    904 flatbuffer_export.cc:4160] Estimated count of arithmetic ops: 2.353 M ops, equivalently 1.177 M MACs
/home/lakshmeesh/.local/lib/python3.10/site-packages/tensorflow/lite/python/interpreter.py:457: UserWarning: Warning: tf.lite.Interpreter is deprecated and is scheduled for deletion in TF 2.20. Please use the LiteRT interpreter from the ai-edge-litert package. See the [migration guide](https://ai.google.dev/edge/litert/migration) for details.
  warnings.warn(_INTERPRETER_DELETION_WARNING)
fully_quantize: 0, inference_type: 6, input_inference_type: INT8, output_inference_type: INT8
I0000 00:00:1780678362.811992    904 flatbuffer_export.cc:4160] Estimated count of arithmetic ops: 2.353 M ops, equivalently 1.177 M MACs
W0000 00:00:1780678362.812054    904 flatbuffer_export.cc:3715] Skipping runtime version metadata in the model. This will be generated by the exporter.
Success! Saved as gtcn_lkp_quantized.tflite
```

Observations and inferences:

- The terminal printed `fully_quantize: 0`; which means that it actually failed quantisation.
- The “quantized” file size is larger than the non-quantised tflite file (org = 1685KB; after “qtz”= 1772KB)
- The estimated MACs for each frame = 1.17M. This is just under Pico’s default rate (133MHz). That is, it may be tough to run this model in real-time after adding other application elements (STFT pre-processor, ADC-DMA streaming to PC / shell interface). This can be solved either by increasing the clock-rate or by moving to a better microcontroller.

Ideally, if we are able to fully int8 quantise it, the model size should be around 400KB, which can be easily fitted on a Pico.

In the current scenario, the easier thing to do would be to convert the .tflite or .onnx file into c/c++ codes using compilers like edge impulse EON, Apache microTVM, and more.